University of Sheffield

A World TOP 100 University

WHAT UNI? STUDENT CHOICE Awards 2024 UNIVERSITY OF THE YEAR

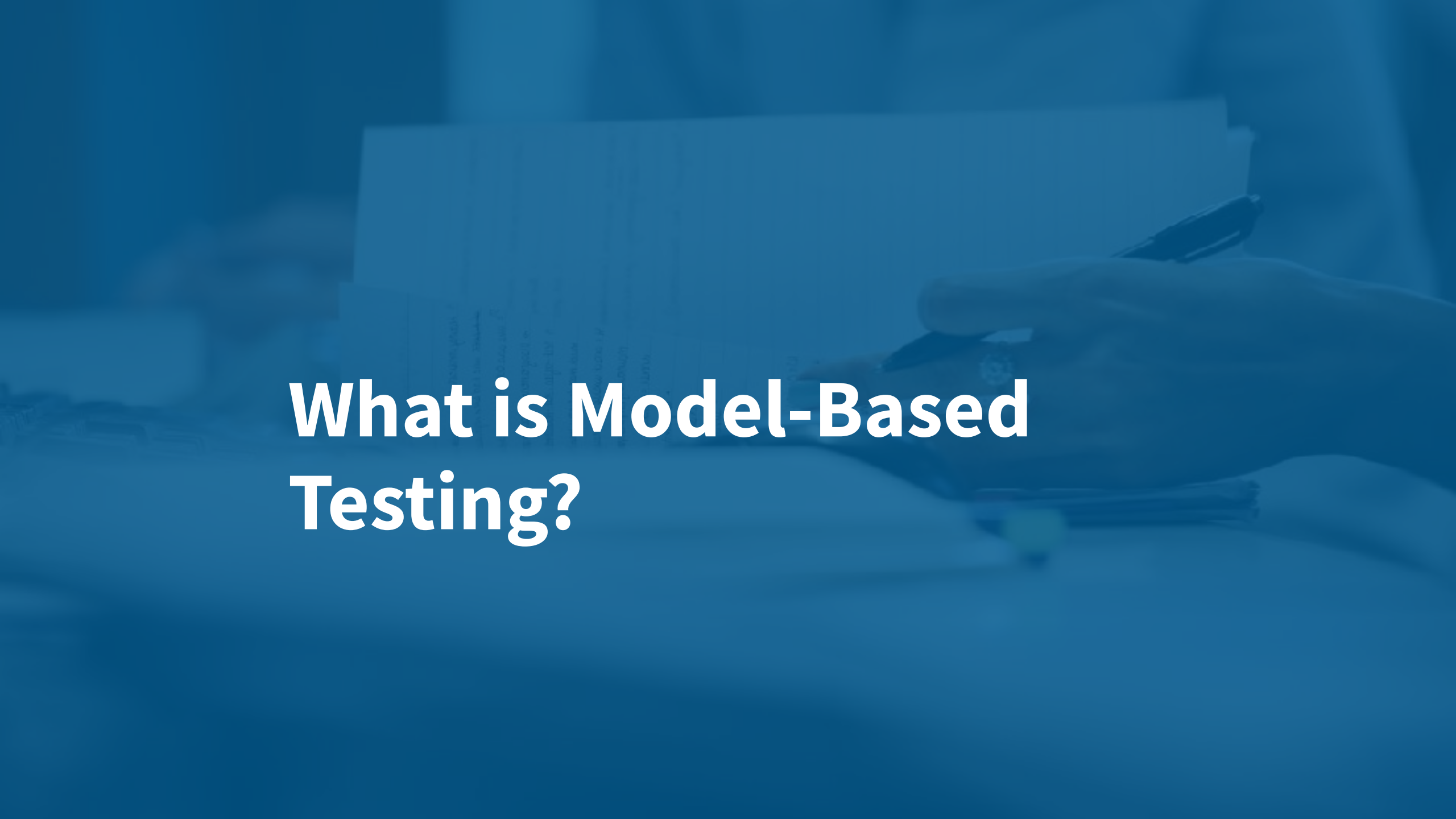
Model-Based Testing

Software Testing (COM3529)

Checkin code: XX-XX-XX

José Rojas & Neil Walkinshaw

Week 7



What is Model-Based Testing?

Models of Software Behaviour

Formally capture ***intended* system behaviour**.

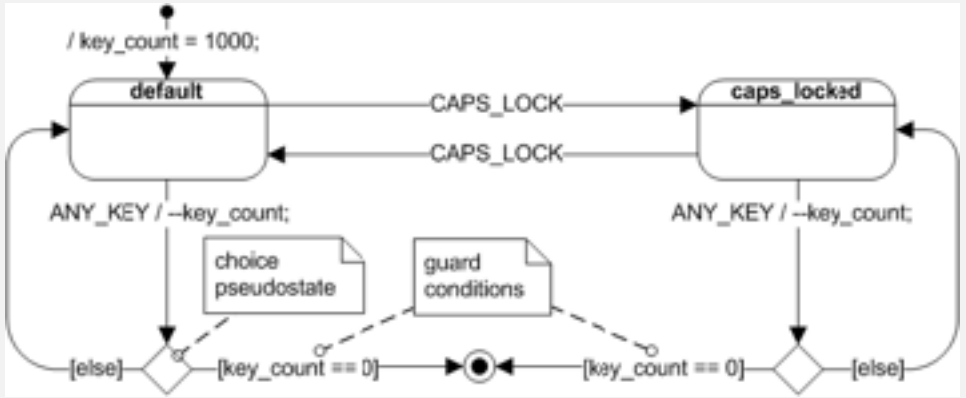
- Abstractions of the low-level source code that capture key behaviours.
- Sometimes even the main development artefact (Model-Based Development)

Many different types of models. E.g.:

- **State-based models** - constraints on the data-state before / after a function has executed.
- **Transition-based models** - potential sequencing of inputs (sometimes corresponding outputs).

```
procedure Increment (X : in out Counter_Type)
  with Global  => null,
       Depends => (X => X),
       Pre     => X < Counter_Type'Last,
       Post    => X = X'Old + 1;
```

Example of pre-/ post-conditions in SPARKAda



Example of a UML state machine

Models are extremely valuable for *testing*

Consider the key testing challenges:

Find inputs that explore key behaviours. **Models explicitly specify possible inputs.**

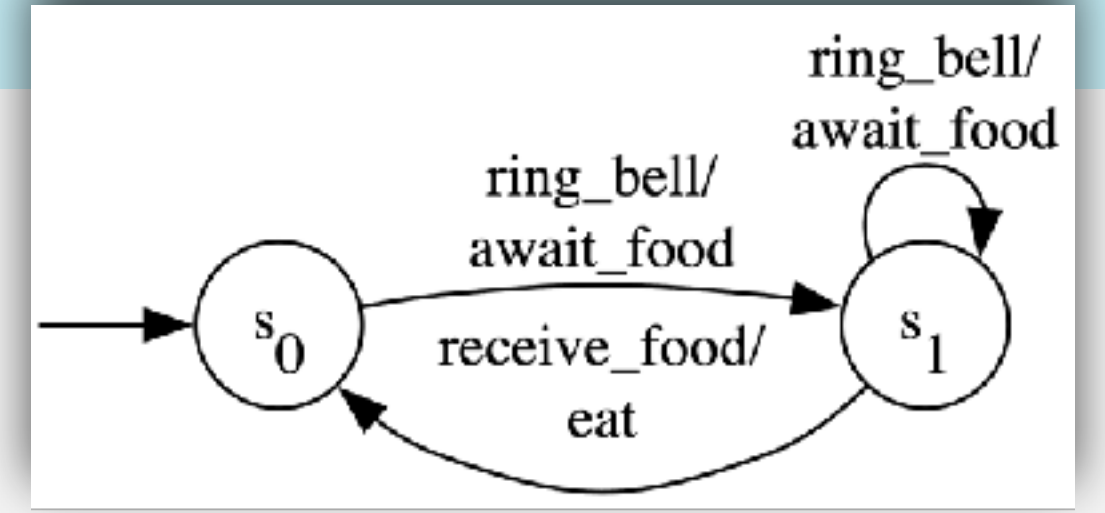
Determine whether or not outputs are correct. **Models specify what outputs to expect.**

Decide when we have tested enough. **Models offer basis for assessing coverage.**



Finite State Machines (FSMs)

Definition of a Finite State Machine



An FSM is a sextuple $(S, s_0, X, Y, \delta, \lambda)$ where:

S is a finite set of states

$s_0 \in S$ is the initial state

X is the input alphabet

$\delta : (S, X) \rightarrow S$ is the state transition function

Y is the output alphabet

$\lambda : (S, X) \rightarrow Y$ is the output function

Can extend δ and λ to take sequences, giving δ^* and λ^*

$S = \{s_0, s_1\}$

$s_0 \in S$ is the initial state

$X = \{ring_bell, receive_food\}$

$$\delta = \begin{cases} (s_0, \text{ ring_bell}) \mapsto s_1 \\ (s_1, \text{ ring_bell}) \mapsto s_1 \\ (s_1, \text{ receive_food}) \mapsto s_0 \end{cases}$$

$Y = \{await_food, eat\}$

$$\lambda = \begin{cases} (s_0, \text{ ring_bell}) \mapsto await_food \\ (s_1, \text{ ring_bell}) \mapsto await_food \\ (s_1, \text{ receive_food}) \mapsto eat \end{cases}$$

Properties of FSMs

An FSM is **deterministic** if there is at most one transition from each state for each input.

This is implicit, since δ is a function.

An FSM is **completely specified** if there is at least one transition from each state for each input.

We can make any FSM complete with an “error state” or “null actions”.

An FSM is **minimal** if there is no trace-equivalent FSM with fewer states.

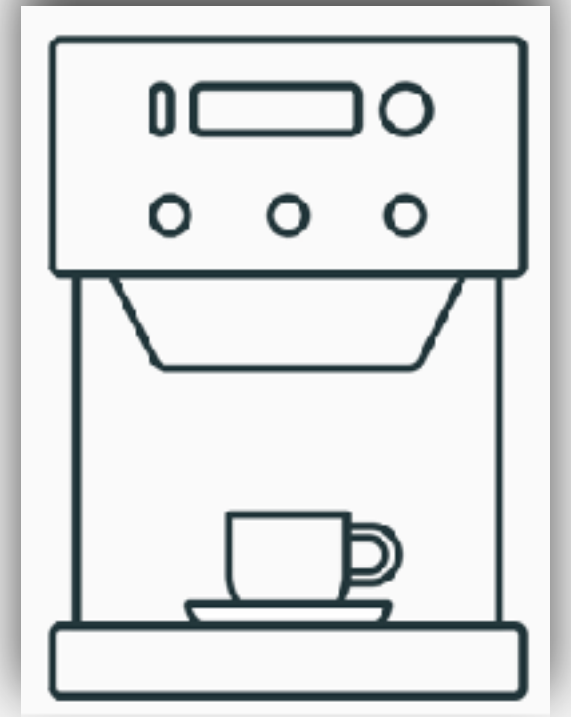
I.e. No smaller FSM in terms of number of states defines the same regular language.

Note, it is possible to convert any FSM into an equivalent minimal FSM.

An FSM is **strongly connected** if, for each pair of states (s_i, s_j) there exists an input sequence that leads from s_i to s_j.

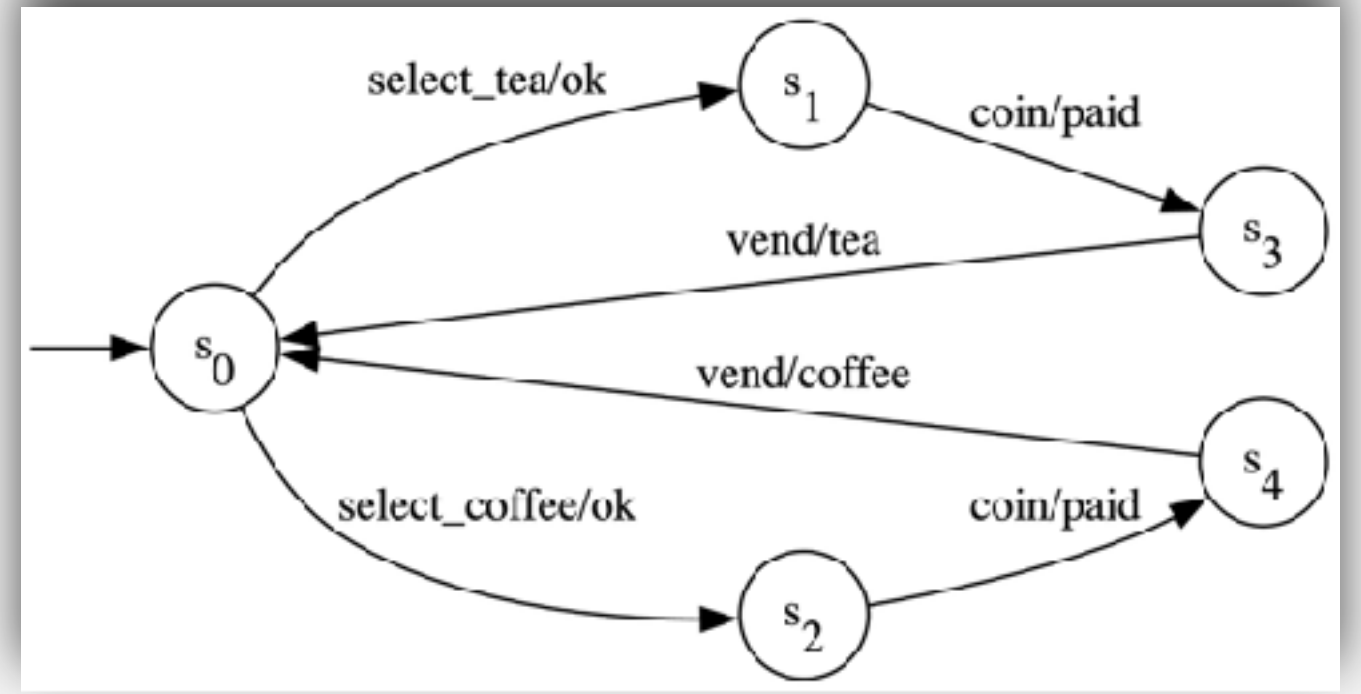
Testing with FSMs

FSM Testing of a Coffee (or Tea) Vending Machine



?

=



Assume that the system behaves like an FSM model.

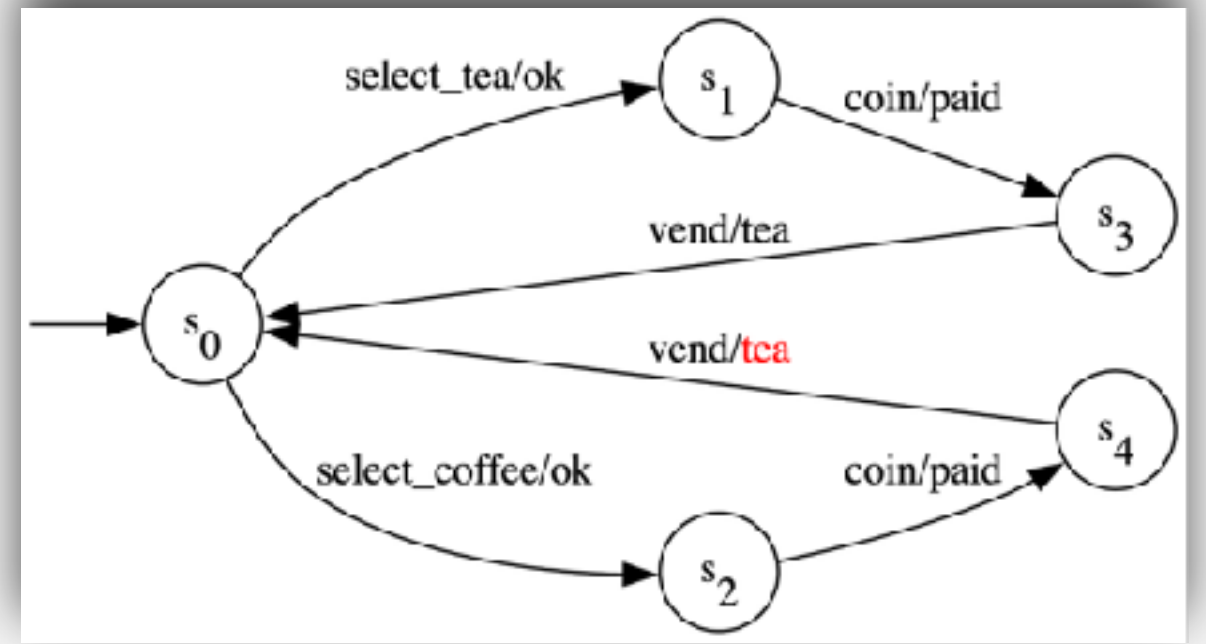
Use our model to derive a test set (a set of input sequences).

Observe corresponding behaviour in the system, and compare outputs.

Output faults

An **output fault** occurs when a transition produces the wrong output.

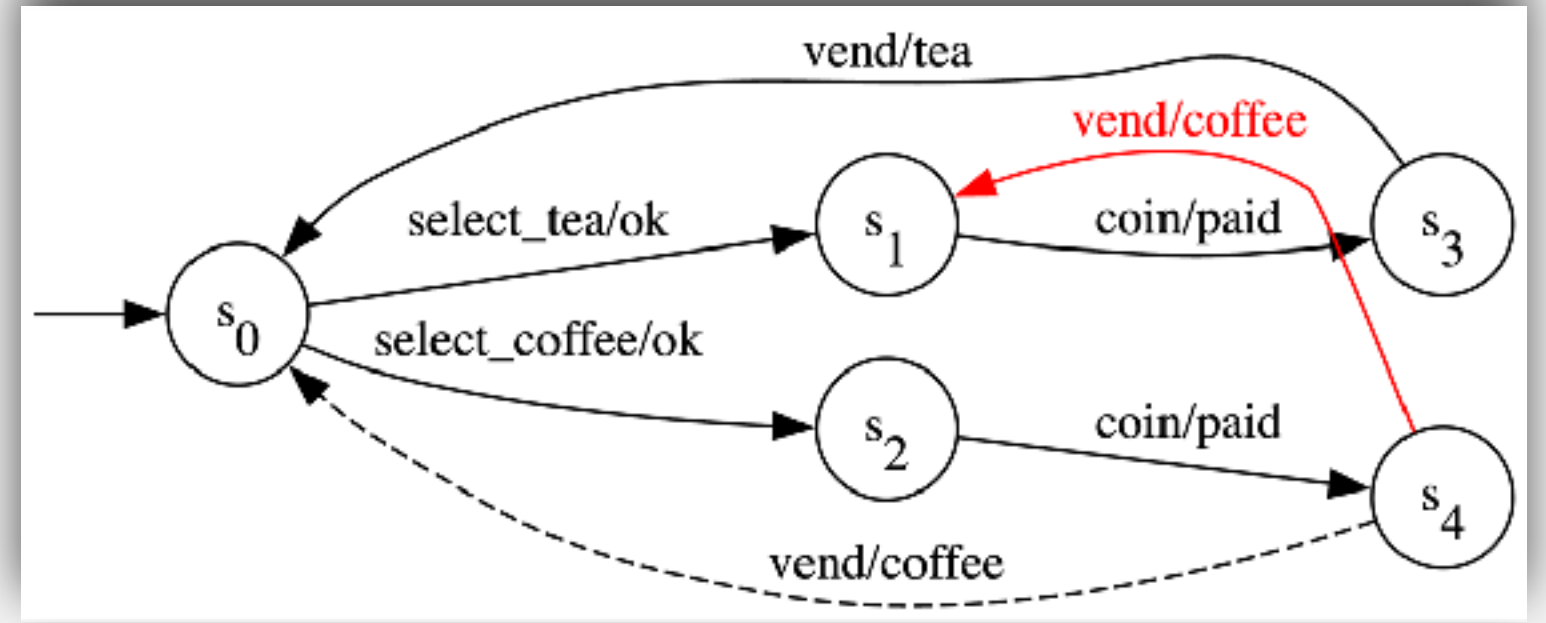
Often the easiest to find - just need to execute the faulty transition to observe an erroneous output.

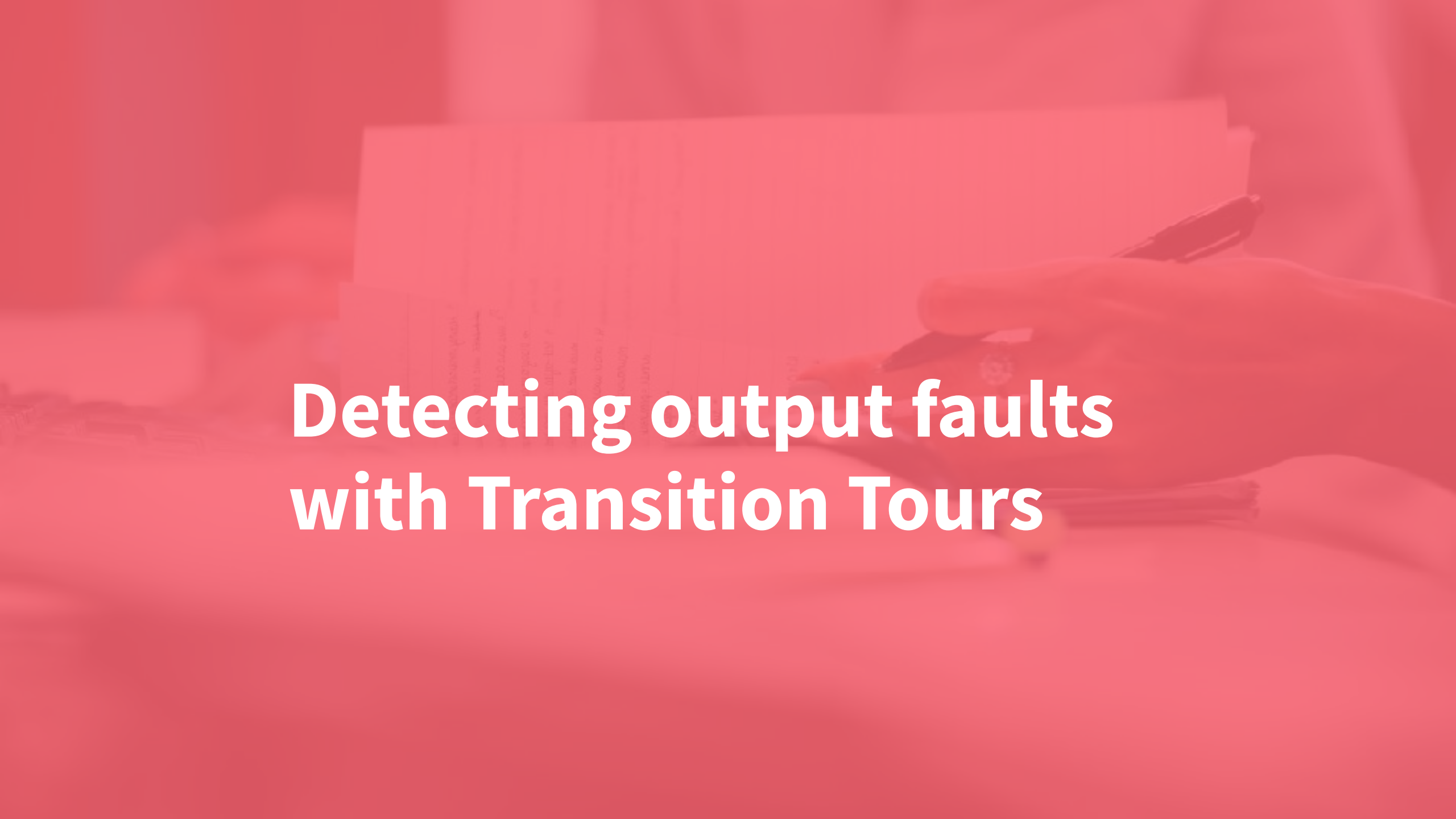


State transfer faults

A **transition fault** occurs when transition leads to the wrong state.

Can also indicate **extra states** in the implementation that do not exist in the model





Detecting output faults with Transition Tours

The Transition Tour Method

Find a path from s_0 that includes **all transitions** in the model.

- If the FSM is strongly connected, this can always be achieved in a single path.
- Otherwise it might require a set of paths.

A **test sequence** is the sequence of inputs and outputs that corresponds to a path.

- During a test, we ensure (1) that the input is possible, and (2) that the output from the system matches the output in the model.

Guaranteed to find all output faults, as long as there are no state transfer faults.

Plenty of graph-theoretical algorithms that can efficiently identify a tour.

- Finding the transition tour is equivalent to the “Chinese Postman Problem” in Graph Theory.

Heuristic approach to obtain Transition Tour

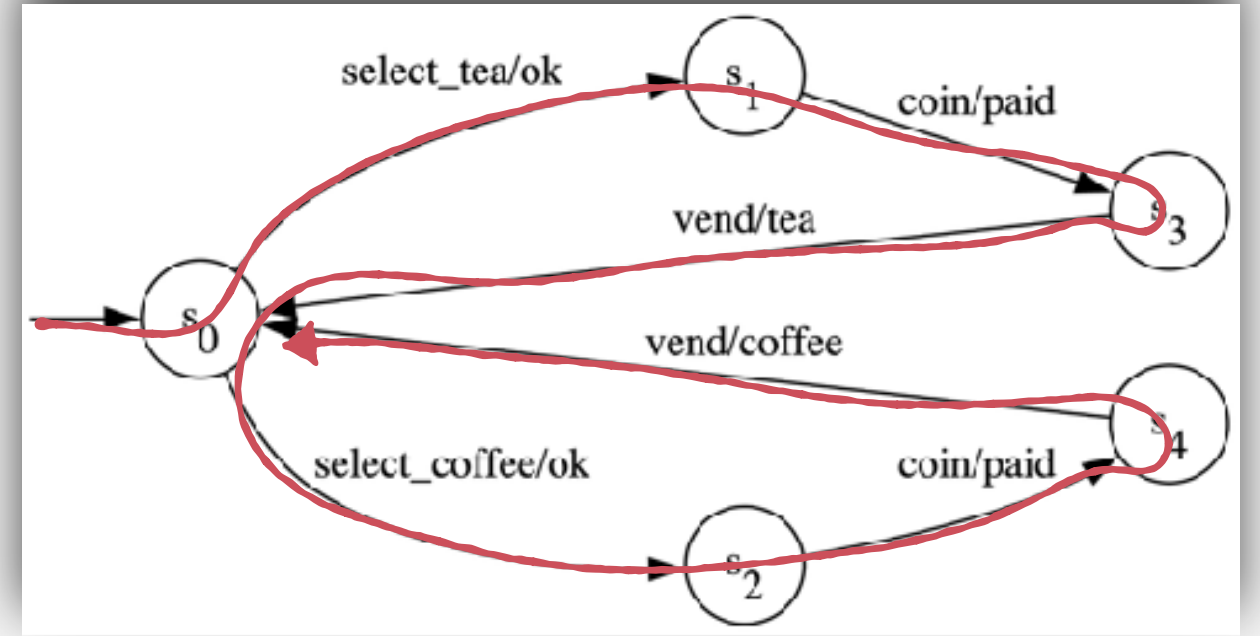
Heuristic approach:

In current state s, if there is a transition t that has not been included:

add t to current path and update state.

Otherwise follow shortest path from s that reaches a state with at least one uncovered transition.

Testing the Drinks Machine

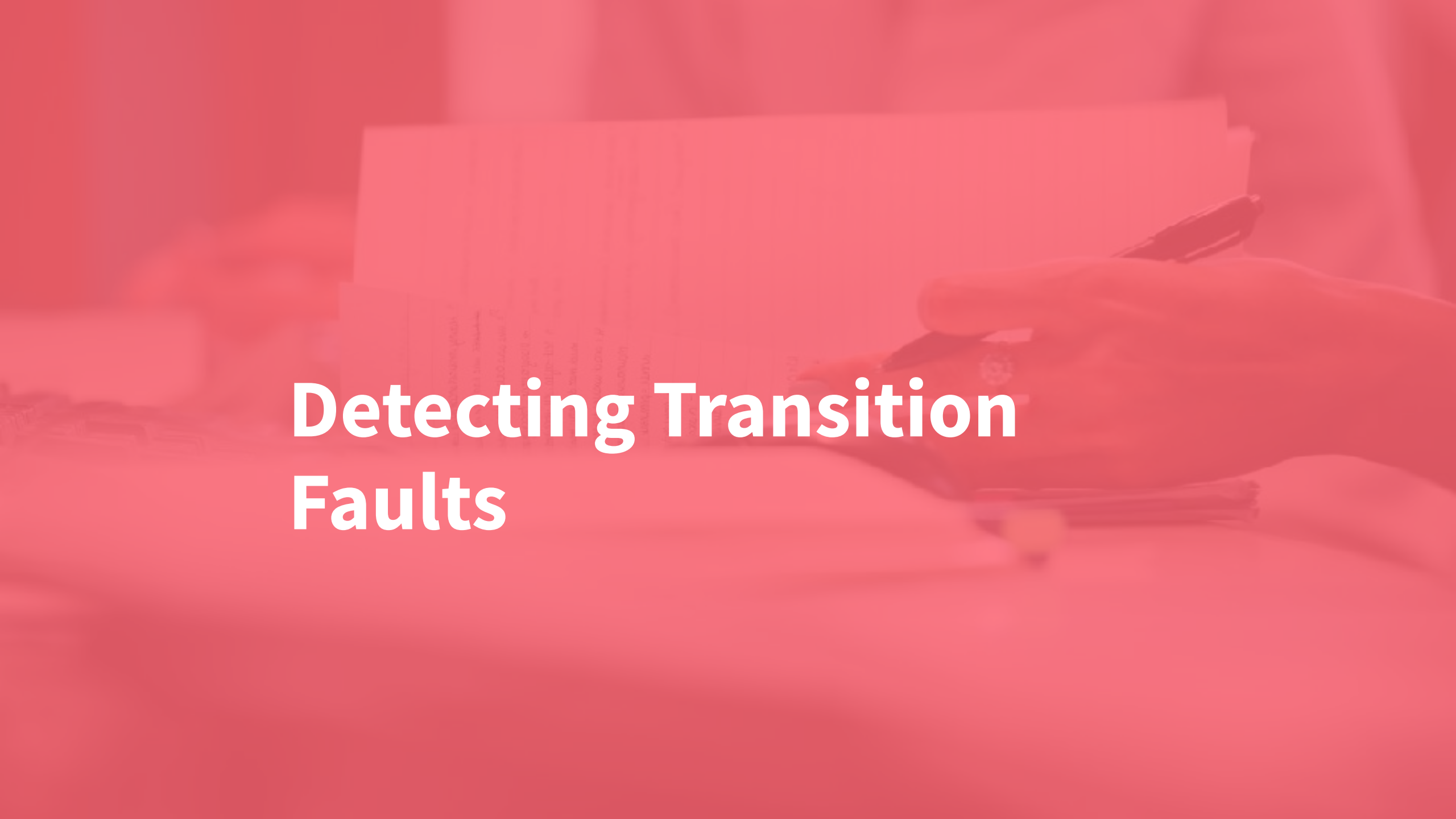


Test sequence:

$select_tea/ok, coin/paid, vend/tea, select_coffee/ok, coin/paid, vend/coffee$

Input sequence: $select_tea, coin, vend, select_coffee, coin, vend$

Output sequence: $ok, paid, tea, ok, paid, coffee$



Detecting Transition Faults

Testing Challenge

To test for transition faults, we want to check every transition $(q_i, q_j, x/y)$.

Basic process:

- Get the system into state q_i
- Submit input x
- Check that we get output y
- **Check that we have ended up in state q_j**

Essential challenge:

- Given that we can only supply inputs and observe outputs …
- … how can we be sure that the system is actually in state q_j?

Distinguishing sequences
Unique I/O sequences (UIOs)
Characterising set

What makes a pair of states different?

The same sequence of inputs returns different outputs.

Formally, two states s and s' are separated by an input sequence x if:

$$(\lambda^*(s, x) \neq \lambda^*(s', x))$$

If there is such a sequence x, then s and s' are said to be **separable**.

If two states are not separable, they are said to be **equivalent**.

Distinguishing Sequences

A distinguishing sequence D is an input sequence, producing different outputs **for each state**.

For every pair of states s and s' such that $s \neq s'$, we have that $(\lambda^*(s, D) \neq \lambda^*(s', D))$

This means that the output produced for D **identifies** the state of the FSM.

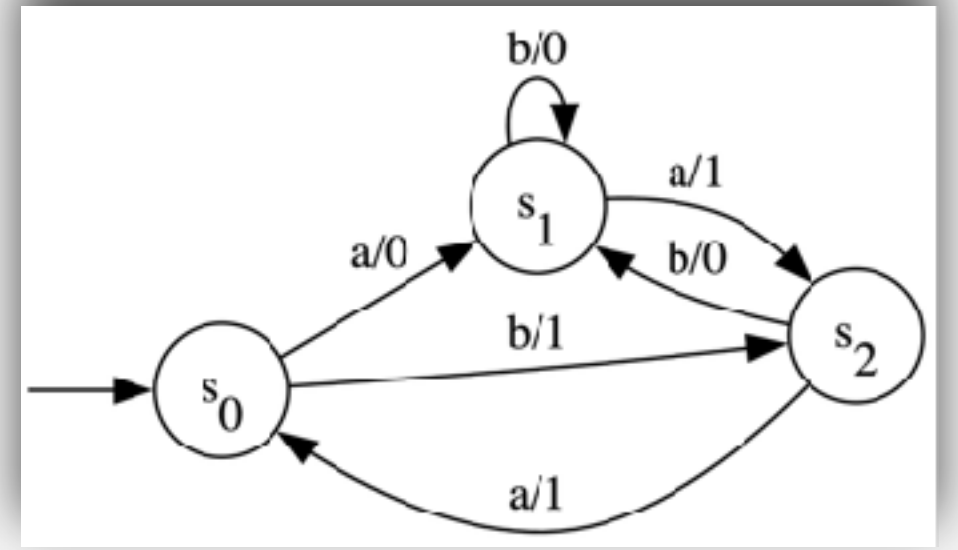
A FSM can have multiple distinguishing sequences, but does not necessarily have one.

For the example machine, we have $D = < a, a >$

From s_0 the output sequence is $< 0,1 >$

From s_1 the output sequence is $< 1,1 >$

From s_2 the output sequence is $< 1,0 >$



Unique I/O Sequences (UIOs)

A UIO sequence separates **one state** from **all other states**.

To see if the FSM is in state s we can execute the UIO sequence for s.

Formally, $\lambda^*(s, \bar{x}) = \bar{y} \implies (\forall s' \,.\, s' \neq S \implies \lambda^*(s', \bar{x}) \neq \bar{y})$

$\bar{x}$ and $\bar{y}$ represent a sequence of inputs and outputs respectively.

If $\bar{x}$ produces $\bar{y}$ then we *must* have been in state s, otherwise we *must* have been in a different state.

Whereas a distinguishing sequence can identify *all* states, a UIO can only identify a single state.

Not all FSMs have UIOs either.

Brute-Force Approach

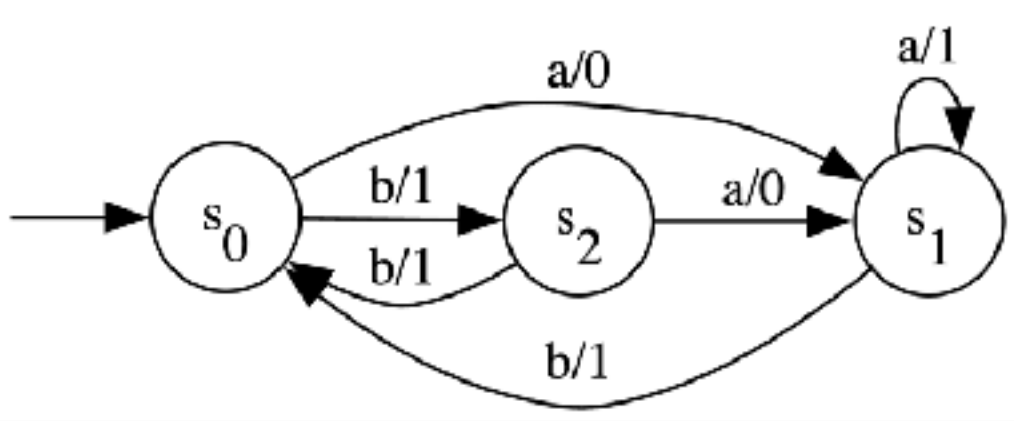
Systematically try out all possible input sequences of increasing length until UIO sequence is found for each state.

- Steps:
  - Start with input sequences of length 1, e.g. ‘a’, ‘b’
  - Apply each sequence to the FSM, starting in each state
    - Building a state / transition table can be helpful for this.
  - Observe output sequences for each state.
    - If unique for a given state with respect *all* other states, store as the UIO for that state.
  - Increase length and repeat until all states have a UIO.

Easy to understand and implement but …

- Computationally expensive for large FSMs with large alphabets.
- Finding UIOs is not guaranteed.

Example



Following the brute-force algorithm (building table from left to right):

State	a	b	aa	ab	ba	bb	aaa	...
s0	0	1	01	01	10	11	011	...
s1	1	1	11	11	10	11	111	...
s2	0	1	01	01	10	11	011	...

In the first iteration, we already identify ‘a’ as a UIO for s1.

‘a’ distinguishes s_1 from all other states.

Up to length 2 we cannot find UIOs for states s_0 and s_2.

Characterising Sets

A **characterising set** W is a set of input sequences that distinguishes each **pair of states**.

For every pair of states s, s' such that $s \neq s'$, we have some $\bar{x} \in W$ such that:
$\lambda^*(s, \bar{x}) \neq \lambda^*(s', \bar{x})$

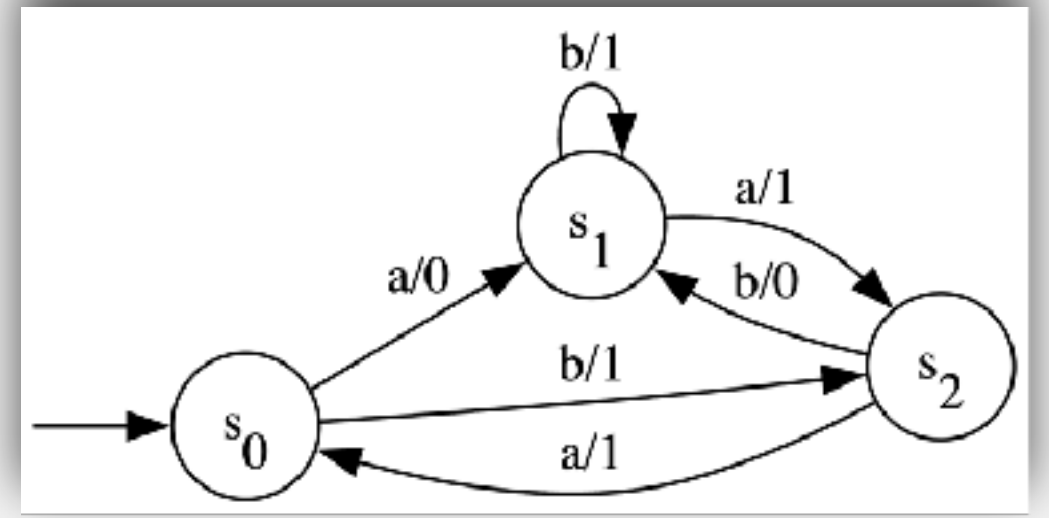
i.e. the input sequence $\bar{x}$ distinguishes these states.

Every minimal FSM with n states has a characterising set W with at most $n - 1$ sequences, of length at most $n - 1$.

There are polynomial time algorithms to generate W sets.

We can minimise every FSM.

Characterising set example



State	Response to a	Response to b
s0	0	1
s1	1	1
s2	1	0

For the above FSM, $W = \{a, b\}$.

If you look at the outputs in response to a and b, they differ for each state.

You can see this in the table.

I.e. For any transition, if we want to check that we are in the correct state, we'll need two tests.

The W-Method

We can build a **fully-fledged test set** that can determine whether a system is equivalent to any FSM.

Concept: For every state s

- Get FSM into state s by supplying the inputs required to get from s_0 to s.
- Use W-set to verify that s in the system under test corresponds to s in the FSM model.
- Try every input $x \in X$ from this state.
- Ensure that the target state $s \xrightarrow{x} s'$ is equivalent to s' in the FSM model.

Computing the W-Method Test Set

Start by building a set V, containing for each state a sequence of inputs that reaches it from s_0.

This must also include the empty sequence ϵ (so that we are 'reaching' s_0 itself)

Compute the set $VW \cup VXW$

VW checks that each input sequence $\bar{x} \in V$ goes to the right state.

VXW checks that every transition from every state goes to the right state.

If the system has at most one extra state, this will be identified by this test set.

To test for m additional test sets, we need to try every possible combination of m inputs.

So we get $VW \cup VXW \cup VX^2W \cup \ldots \cup VX^mW$

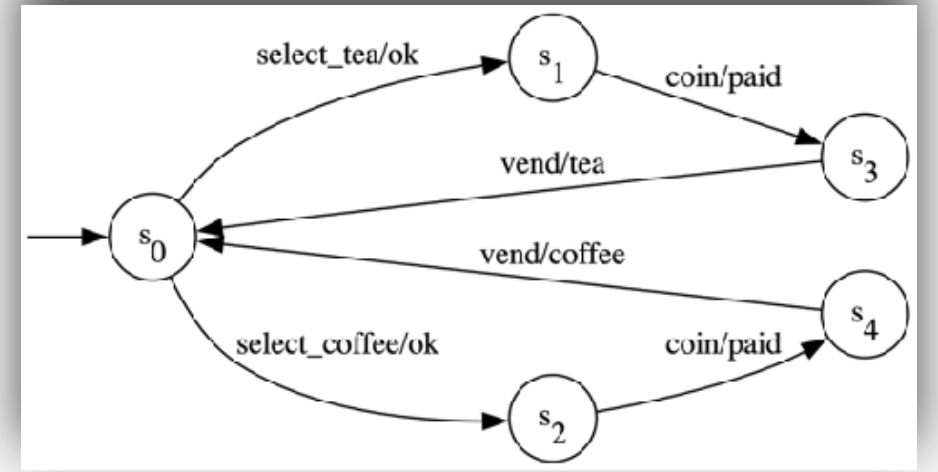
Size of the test set grows exponentially with m.

Testing the Coffee (or Tea) Dispenser with the W-Method

$V = \{\epsilon, < select_tea >, < select_coffee >, < select_tea, coin >, < select_coffee, coin > \}$

$X = \{ < select_tea >, < select_coffee >, < coin >, < vend > \}$

$W = \{ < select_tea, coin, vend > \}$



VW	V	W	Full sequence
1	ε	<select_tea, coin, vend>	<select_tea,coin,vend>
2	<select_tea>		<select_tea, select_tea, coin, vend>
3	<select_coffee>		<select_coffee, select_tea, coin, vend>
4	<select_tea, coin>		<select_tea, coin, select_tea, coin, vend>
5	<select_coffee, coin>		<select_coffee, coin, select_tea, coin, vend>

VXW	V	X	W	Full sequence
1	ε	<select_tea>		<select_tea,coin,vend>
2	ε	<select_coffee>		<select_coffee, select_tea, coin, vend>
3	ε	<coin>		<coin, select_tea, coin, vend>
4	ε	<vend>		<vend, select_tea, coin, vend>
5	<select_tea>	<select_tea>		<select_tea, select_tea, select_tea, coin, vend>
6	<select_tea>	<select_coffee>		<select_tea, select_coffee, select_tea, coin, vend>
7	<select_tea>	<coin>		<select_tea, coin, select_tea, coin, vend>
8	<select_tea>			end, select_tea, coin, vend>
9	<select_coffee>	<s		ect_tea, select_tea, coin, vend>
10	<select_coffee>	<sel		t_coffee, select_tea, coin, vend>
11	<select_coffee>			coin, select_tea, coin, vend>
12	<select_coffee>			vend, select_tea, coin, vend>
13	<select_tea, coin>	<select_tea>		<select_tea, coin, select_tea, select_tea, coin, vend>
14	<select_tea, coin>	<select_coffee>		<select_tea, coin, select_coffee, select_tea, coin, vend>
15	<select_tea, coin>	<coin>		<select_tea, coin, coin, select_tea, coin, vend>
16	<select_tea, coin>	<vend>		<select_tea, coin, vend, select_tea, coin, vend>
17	<select_coffee, coin>	<select_tea>		<select_coffee, coin, select_tea, select_tea, coin, vend>
18	<select_coffee, coin>	<select_coffee>		<select_coffee, coin, select_coffee, select_tea, coin, vend>
19	<select_coffee, coin>	<coin>		<select_coffee, coin, coin, select_tea, coin, vend>
20	<select_coffee, coin>	<vend>		<select_coffee, coin, vend, select_tea, coin, vend>

VW contains 5 tests
VXW contains 20 tests
Full test set contains all 25 tests

Wrapping up

Model Based Testing - So What?

Enormously powerful - can use **testing to guarantee that your system is correct**.

- Provides a solution to the **oracle problem** - are the outputs produced by my tests correct?
- Provides a solution to the **input generation** problem - which inputs to pick.

Subject to several major caveats.

- The model itself needs to be up to date and correct.
- You need to make assumptions about the system.
  - Number of additional states in the system, the fact that you can reliably reset it.

Where do the models come from?

- Particularly powerful in Model-Driven Development, where models exist anyway.
- Several tools that can reverse-engineer models from input/output observations.